

开源软件基础

Python 进阶知识

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 23, 2017



数据准备

```
$head -n 10 robots.txt # from taobao.com/robots.txt
```

```
User-agent: Baiduspider
```

```
Allow: /article
```

```
Allow: /oshtml
```

```
Allow: /wenzhang
```

```
Disallow: /product/
```

```
Disallow: /
```

```
User-Agent: Googlebot
```

```
Allow: /article
```

```
Allow: /oshtml
```

数据准备

```
$cat index.html # use type in win32
```

```
<html>
  <head>
  </head>
  <body>
    <h1>Title</h1>
    <p> This is a paragraph</p>
    <table>
      <tr>
        <td>A11</td> <td>A12</td> <td>A13</td>
      </tr>
      <tr>
        <td>A21</td> <td>A22</td> <td>A23</td>
      </tr>
      <tr>
        <td>A31</td> <td>A32</td> <td>A33</td>
      </tr>
    </body>
  </html>
```

```
$python3 -m http.server
```

```
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/)
```

我发起疯来，连自己都爬

Outline

- 1 Robots 协议
- 2 License
- 3 Requests: HTTP for Humans
- 4 Tutorial

机器人协议

robots.txt (统一小写) 是一种存放于网站根目录下的 ASCII 编码的文本文件, 它通常告诉网络搜索引擎的漫游器 (又称网络蜘蛛), 此网站中的哪些内容是不应被搜索引擎的漫游器获取的, 哪些是可以被漫游器获取的。因为一些系统中的 URL 是大小写敏感的, 所以 robots.txt 的文件名应统一为小写。robots.txt 应放置于网站的根目录下。如果想单独定义搜索引擎的漫游器访问子目录时的行为, 那么可以将自定的设置合并到根目录下的 robots.txt, 或者使用 robots 元数据 (Metadata, 又称元数据)。¹

¹<https://zh.wikipedia.org/wiki/Robots.txt>

机器人协议

协议特点

- robots.txt 协议并不是一个规范，而只是约定俗成的，所以并不能保证网站的隐私。
- 用字符串比较来确定是否获取 URL，所以目录末尾有与没有斜杠“/”表示的是不同的 URL。
- 允许使用类似"Disallow: *.gif" 这样的通配符。

机器人协议

允许所有的机器人：

User-agent: *

Disallow:

另一写法

User-agent: *

Allow: /

机器人协议

仅允许特定的机器人：

```
User-agent: name_spider  
Allow:
```

拦截所有的机器人

```
User-agent: *  
Disallow: /
```

机器人协议

禁止所有机器人访问特定目录

```
User-agent: *  
Disallow: /cgi-bin/  
Disallow: /images/  
Disallow: /tmp/  
Disallow: /private/
```

仅禁止坏爬虫访问特定目录

```
User-agent: BadBot  
Disallow: /private/
```

机器人协议

禁止所有机器人访问特定目录

```
User-agent: *  
Disallow: /*.php$  
Disallow: /*.js$  
Disallow: /*.inc$  
Disallow: /*.css$
```

如何遵守协议

- 人工阅读 robots.txt
- 使用 urllib.robotparser

解析 robots.txt

```
>>> import urllib.robotparser
>>> rp = urllib.robotparser.RobotFileParser()
>>> rp.set_url("http://127.0.0.1:8000/robots.txt")
>>> rp.read()
>>> rp.can_fetch("Baiduspider", "http://127.0.0.1:8000/")
False
>>> rp.can_fetch("Baiduspider", "http://127.0.0.1:8000/article/")
True
```

Outline

- 1 Robots 协议
- 2 License
- 3 Requests: HTTP for Humans
- 4 Tutorial

Philosophy

All contributions to Requests should keep these important rules in mind.

Apache2 License

A large number of open source projects you find today are GPL Licensed. While the GPL has its time and place, it should most certainly not be your go-to license for your next open source project.

Apache2 License

A project that is released as GPL cannot be used in any commercial product without the product itself also being offered as open source.

Apache2 License

The MIT, BSD, ISC, and Apache2 licenses are great alternatives to the GPL that allow your open-source software to be used freely in proprietary, closed-source software.

Apache2 License

Requests is released under terms of Apache2 License.

Outline

- 1 Robots 协议
- 2 License
- 3 Requests: HTTP for Humans**
- 4 Tutorial

Requests: HTTP for Humans

Requests is the only Non-GMO HTTP library for Python, safe for human consumption.

Requests: HTTP for Humans

Note

Requests: HTTP for Humans

The use of Python 3 is highly preferred over Python 2. Consider upgrading your applications and infrastructure if you find yourself still using Python 2 in production today. If you are using Python 3, congratulations — you are indeed a person of excellent taste. — Kenneth Reitz

Requests: HTTP for Humans

Behold, the power of Requests:

Requests: HTTP for Humans

```
>>> r = requests.get('https://api.github.com/user', au
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf8'
>>> r.encoding
'utf-8'
>>> r.text
u'{"type":"User"...'}
>>> r.json()
{u'private_gists': 419, u'total_private_repos': 77, ..
```

Requests: HTTP for Humans

See similar code, sans Requests.

Requests: HTTP for Humans

Requests allows you to send organic, grass-fed HTTP/1.1 requests, without the need for manual labor. There's no need to manually add query strings to your URLs, or to form-encode your POST data. Keep-alive and HTTP connection pooling are 100% automatic, thanks to urllib3.

User Testimonials

Twitter, Spotify, Microsoft, Amazon, Lyft, BuzzFeed, Reddit, The NSA, Her Majesty's Government, Google, Twilio, Runscope, Mozilla, Heroku, PayPal, NPR, Obama for America, Transifex, Native Instruments, The Washington Post, SoundCloud, Kippt, Sony, and Federal U.S. Institutions that prefer to be unnamed claim to use Requests internally.

User Testimonials

Requests is one of the most downloaded Python packages of all time, pulling in over 13,000,000 downloads every month. All the cool kids are doing it!

Beloved Features

Requests is ready for today's web.

Beloved Features

Requests officially supports Python 2.6–2.7 & 3.4–3.7, and runs great on PyPy.

Outline

- 1 Robots 协议
- 2 License
- 3 Requests: HTTP for Humans
- 4 Tutorial**

发起连接

```
import requests
r = requests.get("http://127.0.0.1:8000")
print(r.text)
print(r.encoding)
print(r.url)
print(r.status_code)
```

GET 参数

```
import requests
import json
payload = {'key1': 'value1', 'key2': 'value2'}
r = requests.get('http://httpbin.org/get', params=payload)
result = json.loads(r.text)
print(result['args'])
```

编码问题

```
r = requests.get("http://127.0.0.1:8000")  
r.encoding = "UTF-8"  
print(r.text)
```